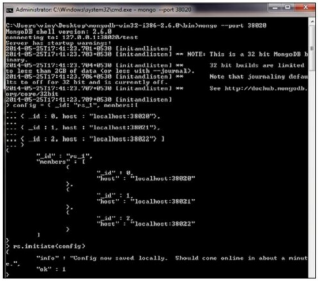

Figure1: Replica set configuration

create the two others.

Open two more command prompts, switch to the MongoDB root directory and type the following commands in each, respectively:

```
mongod --port 38021 --dbpath .\data\shard_1\rs1 --replSet
rs_1 --shardsvr

mongod --port 38022 --dbpath .\data\shard_1\rs2 --replSet
rs_1 --shardsvr
```

Three nodes for the first replica set are ready, but right now they are acting as standalone nodes. We have to configure them to behave as replica sets. Nodes in a replica set maintain the same data and are used for data redundancy. If a node goes down, the deployment will still perform normally. Open a command and switch to the MongoDB root directory, and type:

```
mongo --port 38020.
```

This will start the client process and connect to the server daemon running on Port 38020 that we started earlier. Here, set the configuration as shown below:

```
config = { _id: "rs_1", members:[
{ _id : 0, host : "localhost:38020"},
{ _id : 1, host : "localhost:38021"},
{ _id : 2, host : "localhost:38022"} ]
}
```

Initiate the configuration with the *rs.initiate* (config) command. You can verify the status of the replica set with the *rs.status()* command.

Repeat the same process to create another replica set with the following server information:

```
mongod --port 48020 --dbpath .\data\shard_2\rs0 --replSet
rs_2 --shardsvr
mongod --port 48021 --dbpath .\data\shard_2\rs1 --replSet
rs_2 --shardsvr
mongod --port 48022 --dbpath .\data\shard_2\rs2 --replSet
rs_2 --shardsvr
```

The configuration information is as follows:

```
config = { _id: "rs_2", members:[
{ _id : 0, host : "localhost:48020"},
{ _id : 1, host : "localhost:48021"},
{ _id : 2, host : "localhost:48022"} ]
}
```

Initiate the configuration by using the same *rs.initiate* (config) command. We now have two replica sets, *rs_1* and *rs_2*, up and running; so, the first phase is complete.

Let's now configure our config servers, which are *mongod* instances used to store the metadata related to the sharded cluster we are going to configure. Config servers need to be available for a functional sharded cluster. In our production systems, we use three config servers, but for development and testing purposes, usually one does the job. Here, we'll configure three config servers as per the standard practice. So, first, let's create directories for our three config servers:

```
mkdir .\data\config_1
mkdir .\data\config_1\1  .\data\config_1\2 .\data\config_1\3
Next open 3 more command prompts and type
 mongod --port 59020 --dbpath .\data\config_1\1  --configsvr
mongod --port 59021 --dbpath .\data\config_1\2  --configsvr
mongod --port 59022 --dbpath .\data\config_1\3  --configsvr
```

This will start the three config servers we'll be using for this deployment. The final phase involves configuring the Mongo router or *mongos*, which is responsible for query processing as well as data access in a sharded environment, and is another binary in the Mongo root directory. Open a command prompt and switch to the MongoDB root directory. Type the following command:

```
 mongos --configdb localhost:59020,localhost:59021,localho
st:59022
```

This starts the *mongos* router on the default Port 27017 and informs it about the config servers. Then it's time to add shards and complete the final few steps before our sharding environment is up and running. Open another command prompt and again switch to the MongoDB root directory.

Type *mongo*, which will connect to the *mongos* router on the default Port 27017, and type the following commands: